

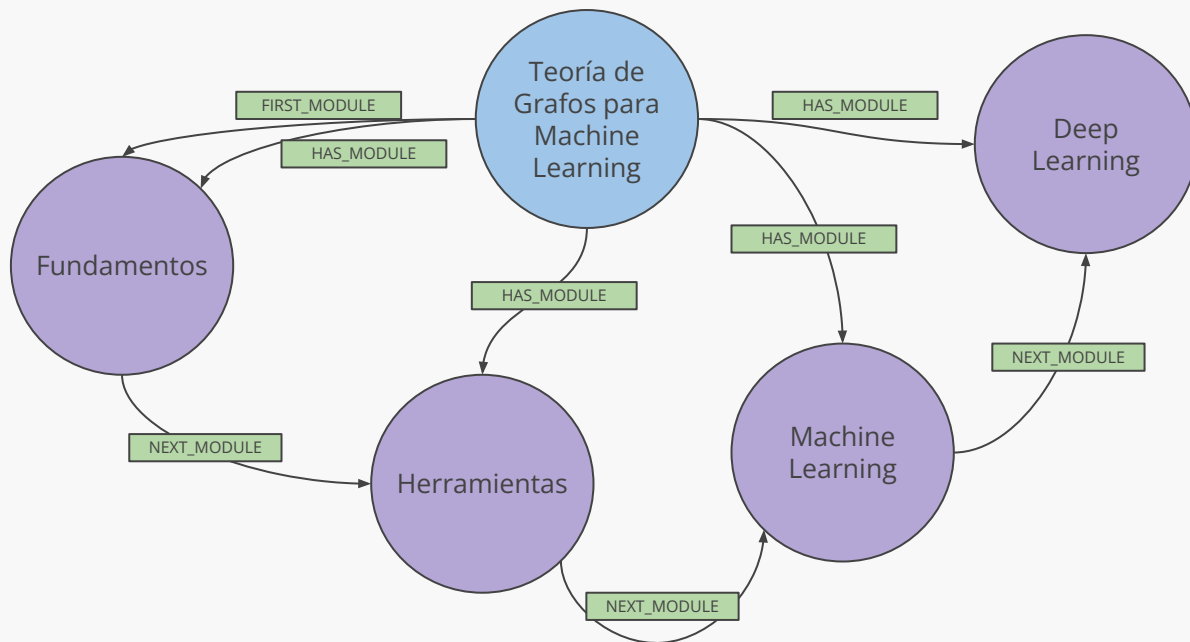
Clase 3

Bases de datos para grafos

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Contenidos del curso

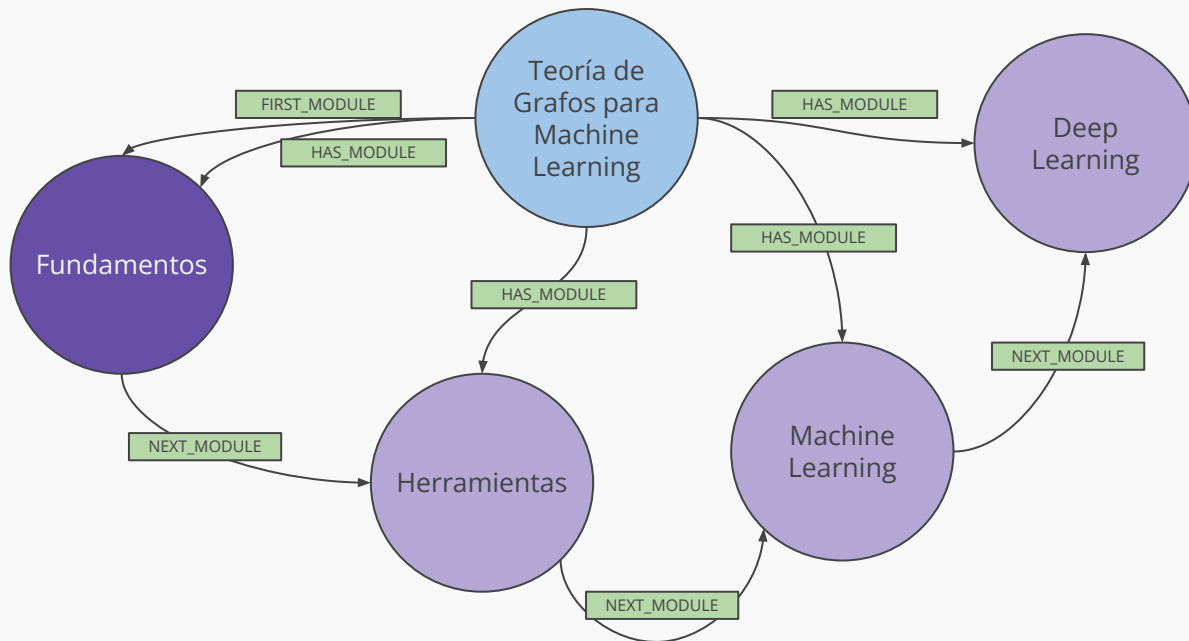


Contenidos del curso

1. Fundamentos

Desde conceptos a **algoritmos fundamentales** en teoría de grafos...

... para establecer una base sólida en los conceptos fundamentales de la teoría de grafos

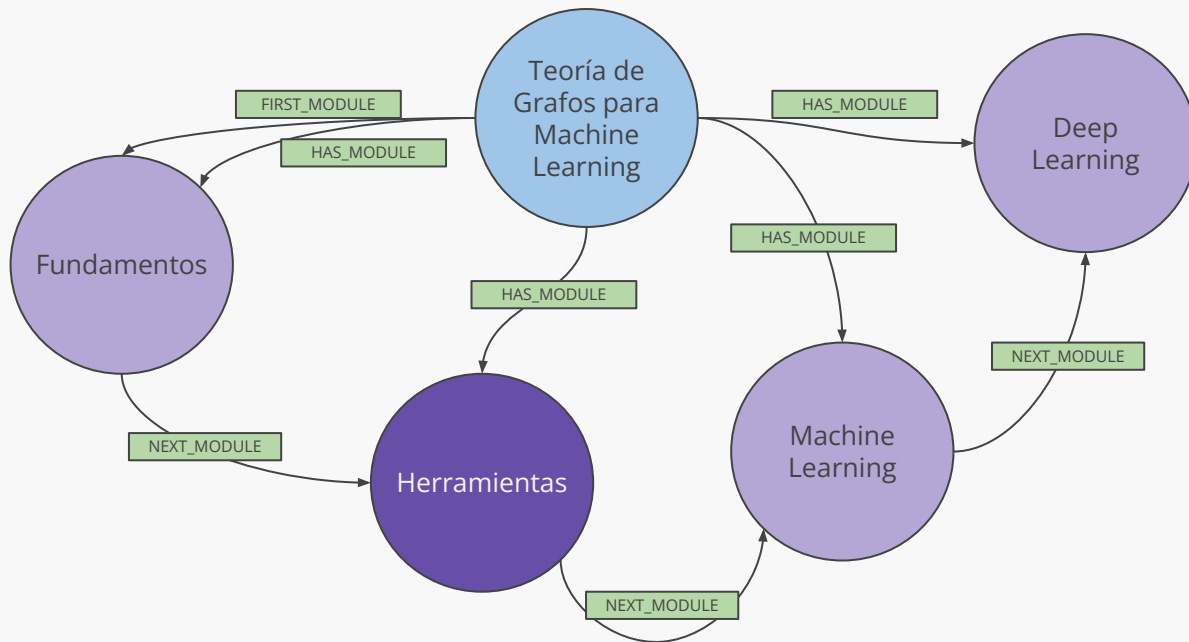


Contenidos del curso

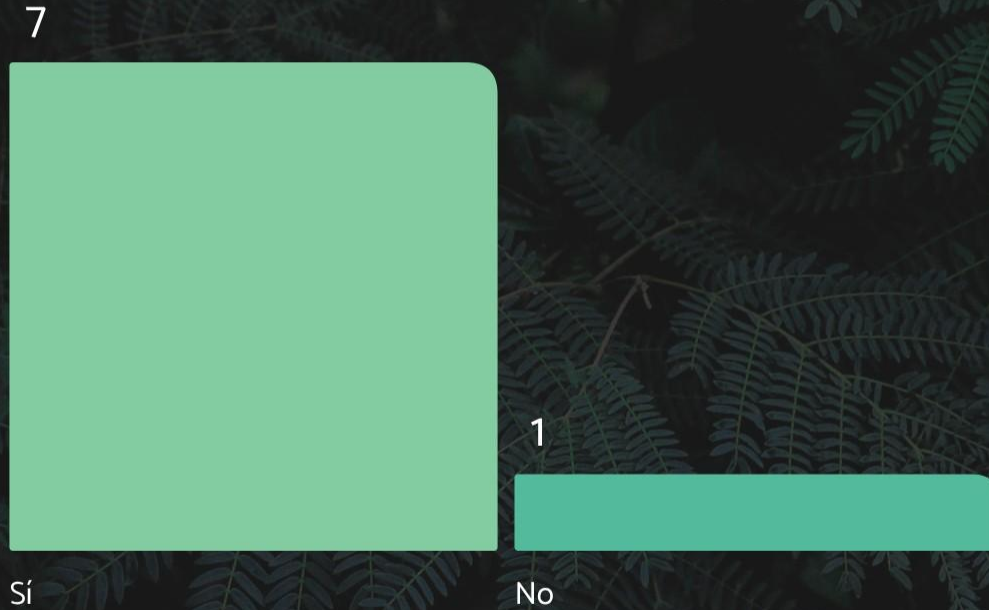
2. Herramientas

Desde representación de pequeños grafos a **análítica de grandes redes...**

... para explorar y utilizar algoritmos fundamentales sobre grafos de cualquier tamaño en distintos contextos



¿Hicieron el control 1?



¿Han ocupado alguna base de datos?



- 0 No he interactuado con grandes cantidades de datos
- 0 Solo he necesitado usar archivos
- 1 He utilizado SQL algunas veces
- 7 Frecuentemente uso bases de datos

¡Gracias por sus respuestas!

Vamos con la clase de hoy

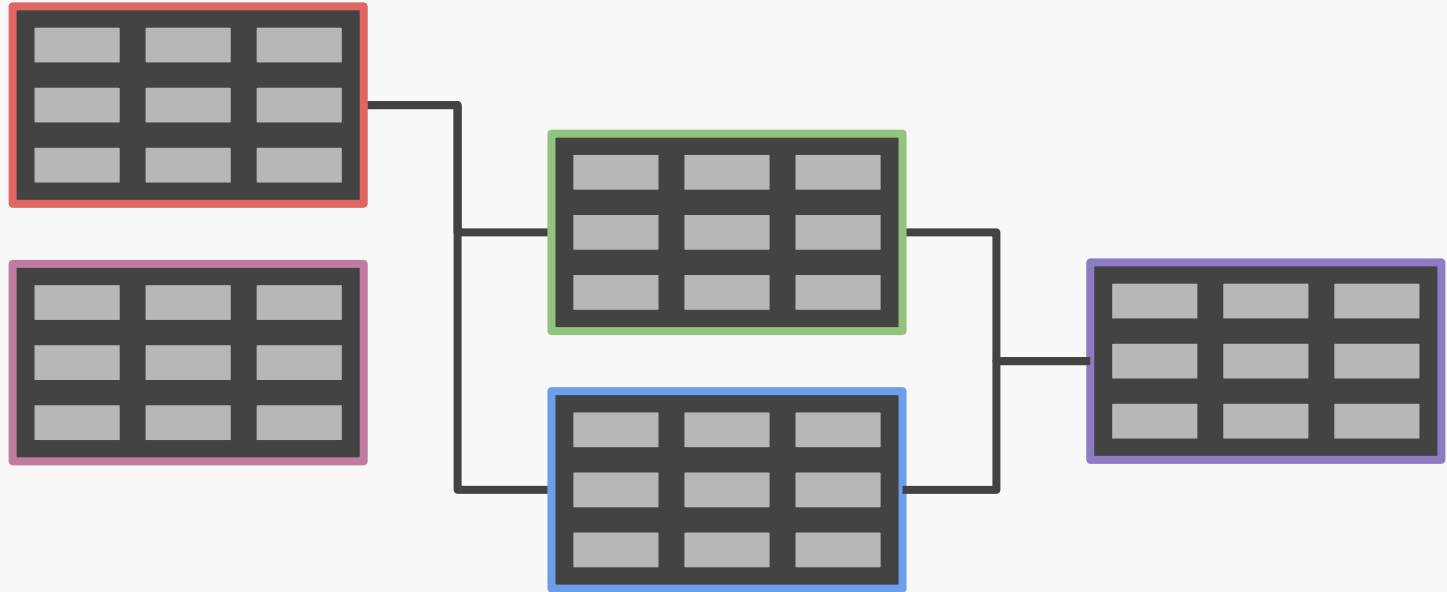
Clase 3.1

Neo4j

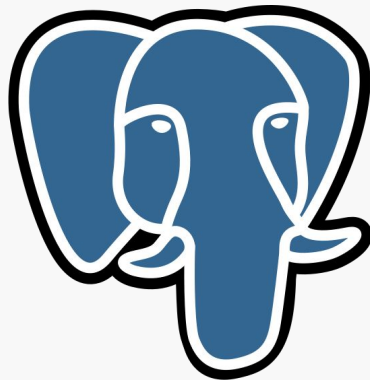
Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl

Bases de datos



Bases de datos



ORACLE



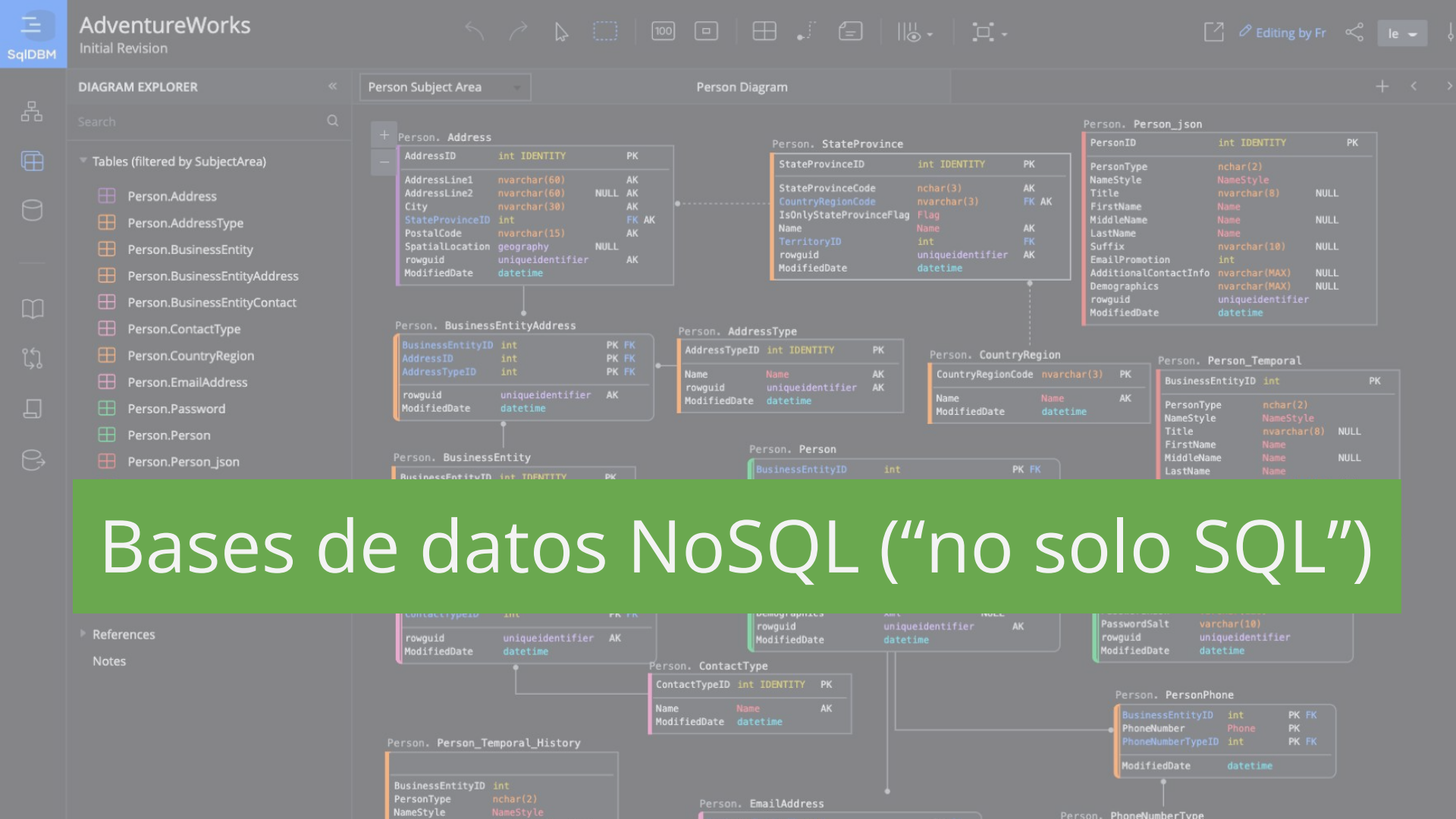
Q

Notes

The diagram illustrates the database schema for AdventureWorks2012, showing the following tables and their attributes:

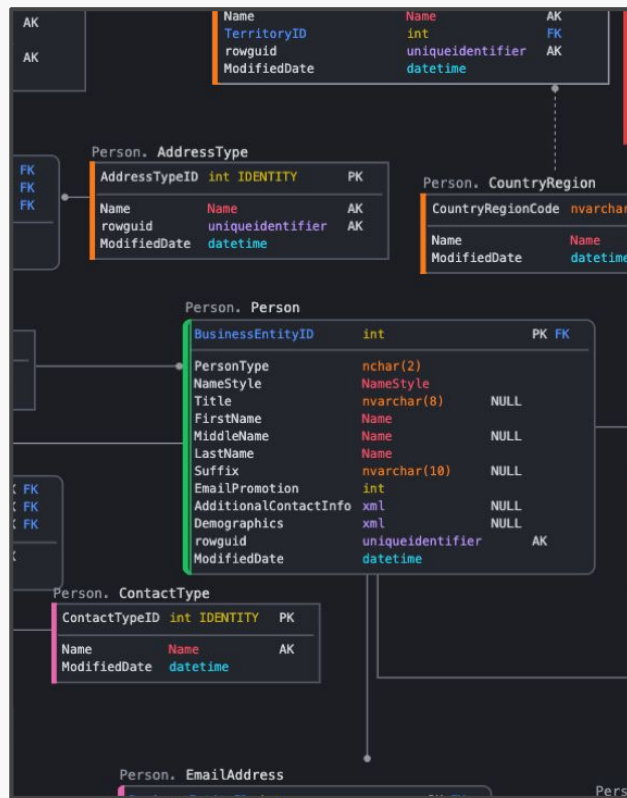
- Person.Address**
 - AddressID (int IDENTITY, PK)
 - AddressLine1 (nvarchar(60), AK)
 - AddressLine2 (nvarchar(60), NULL, AK)
 - City (nvarchar(30), AK)
 - StateProvinceID (int, FK, AK)
 - PostalCode (nvarchar(15), AK)
 - SpatialLocation (geography, NULL)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.StateProvince**
 - StateProvinceID (int IDENTITY, PK)
 - StateProvinceCode (nvarchar(3), AK)
 - CountryRegionCode (nvarchar(3), FK, AK)
 - IsOnlyStateProvinceFlag (Flag, AK)
 - Name (Name, AK)
 - TerritoryID (int, FK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.Person**
 - BusinessEntityID (int, PK, FK)
 - PersonType (nvarchar(2), AK)
 - NameStyle (nvarchar(8), NULL)
 - Title (nvarchar(8), NULL)
 - FirstName (Name, NULL)
 - MiddleName (Name, NULL)
 - LastName (Name, NULL)
 - Suffix (nvarchar(10), NULL)
 - EmailPromotion (int, NULL)
 - AdditionalContactInfo (xml, NULL)
 - Demographics (xml, NULL)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.BusinessEntityAddress**
 - BusinessEntityID (int, PK, FK)
 - AddressID (int, PK, FK)
 - AddressTypeID (int, PK, FK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.AddressType**
 - AddressTypeID (int IDENTITY, PK)
 - Name (Name, AK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.CountryRegion**
 - CountryRegionCode (nvarchar(3), PK)
 - Name (Name, AK)
 - ModifiedDate (datetime)
- Person.Person_Temporal**
 - BusinessEntityID (int, PK)
 - PersonType (nvarchar(2), AK)
 - NameStyle (nvarchar(8), NULL)
 - Title (nvarchar(8), NULL)
 - FirstName (Name, NULL)
 - MiddleName (Name, NULL)
 - LastName (Name, NULL)
 - Suffix (nvarchar(10), NULL)
 - EmailPromotion (int, NULL)
 - ValidFrom (datetime2(7), NULL)
 - ValidTo (datetime2(7), NULL)
- Person.BusinessEntity**
 - BusinessEntityID (int IDENTITY, PK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.BusinessEntityContact**
 - BusinessEntityID (int, PK, FK)
 - PersonID (int, PK, FK)
 - ContactTypeID (int, PK, FK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.ContactType**
 - ContactTypeID (int IDENTITY, PK)
 - Name (Name, AK)
 - ModifiedDate (datetime)
- Person.Password**
 - BusinessEntityID (int, PK, FK)
 - PasswordHash (varchar(128), AK)
 - PasswordSalt (varchar(10), AK)
 - rowguid (uniqueidentifier, AK)
 - ModifiedDate (datetime)
- Person.PersonPhone**
 - BusinessEntityID (int, PK, FK)
 - PhoneNumber (Phone, PK)
 - PhoneNumberTypeID (int, PK, FK)
 - ModifiedDate (datetime)
- Person.Person_Temporal_History**
 - BusinessEntityID (int, PK)
 - PersonType (nvarchar(2), AK)
 - NameStyle (nvarchar(8), NULL)
- Person.EmailAddress**
 - BusinessEntityID (int, PK, FK)
 - EmailAddress (nvarchar(256), AK)
 - ModifiedDate (datetime)
- Person.PhoneNumberType**
 - PhoneNumberTypeID (int IDENTITY, PK)
 - Name (Name, AK)
 - ModifiedDate (datetime)

Relationships are indicated by lines connecting the tables, with crow's foot notation symbols (one-to-many, mandatory, etc.) at the ends of the lines.



Bases de datos NoSQL

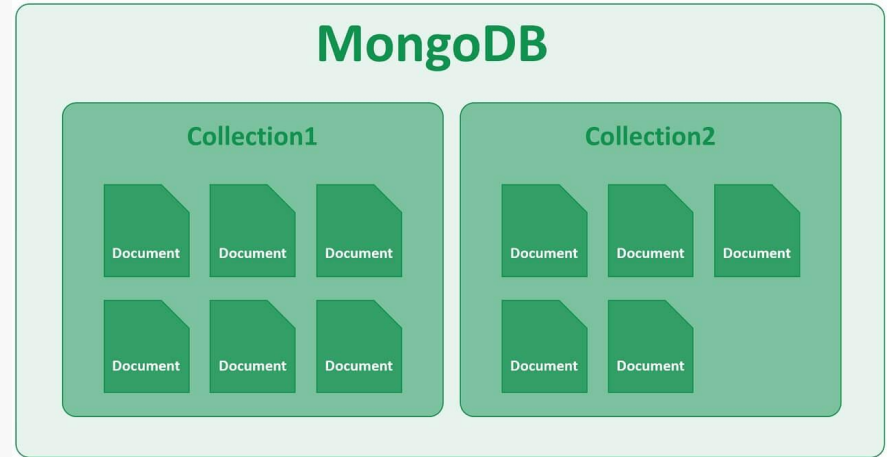
- Menos restricciones que un esquema relacional
- “Schemaless”
- Facilidad para distribuir



Tipos de bases de datos NoSQL



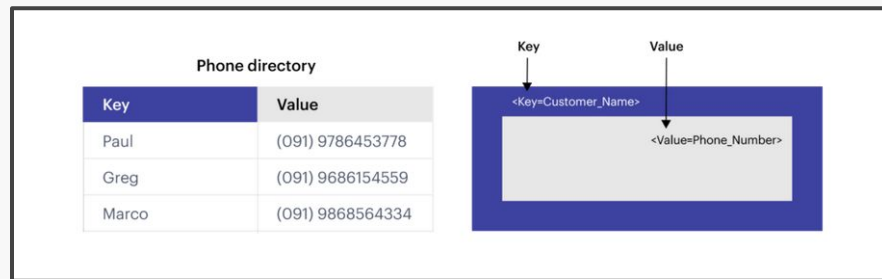
- **Bases de datos de documentos**
- Bases de datos key - value
- Bases de datos vectoriales
- Bases de datos de grafos



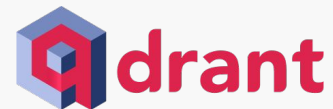
Tipos de bases de datos NoSQL



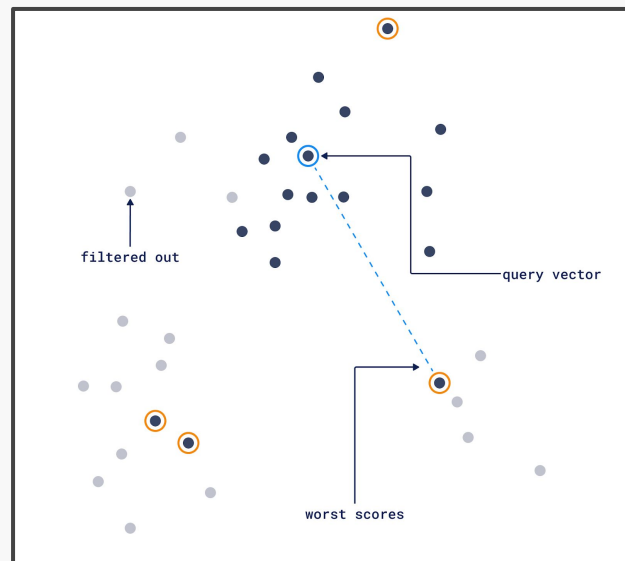
- Bases de datos de documentos
- **Bases de datos key - value**
- Bases de datos vectoriales
- Bases de datos de grafos



Tipos de bases de datos NoSQL



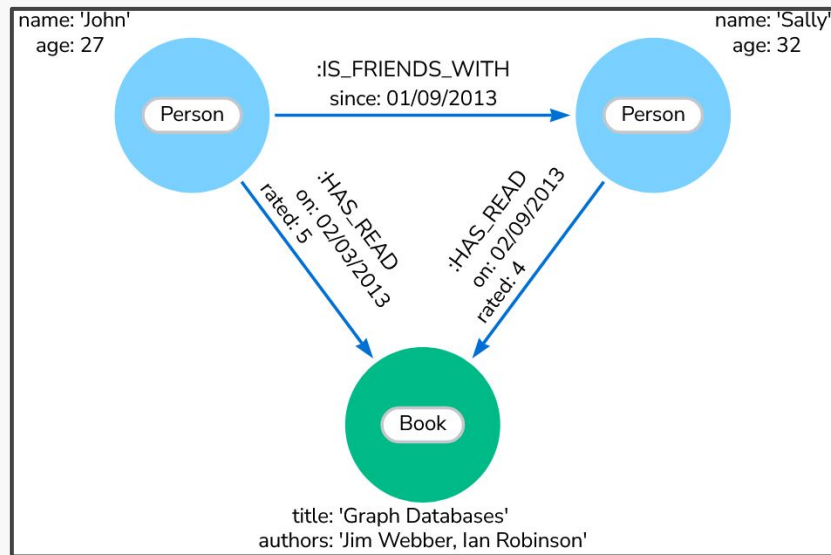
- Bases de datos de documentos
- Bases de datos key - value
- **Bases de datos vectoriales**
- Bases de datos de grafos



Tipos de bases de datos NoSQL



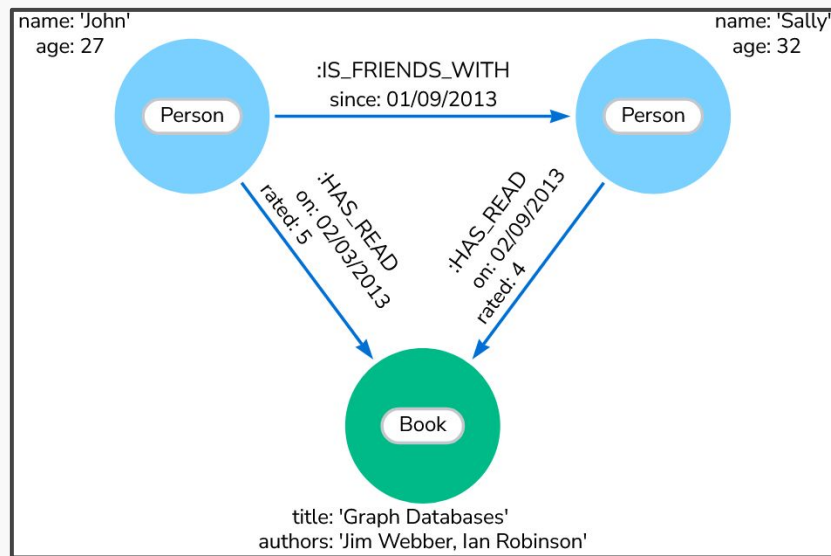
- Bases de datos de documentos
- Bases de datos key - value
- Bases de datos vectoriales
- **Bases de datos de grafos**



¿Y por qué no usar una base de datos relacional?



- Las bases de datos de grafos están orientadas a encontrar relaciones rápidamente
- Las bases de datos relacionales son muy generales, mientras que las de grafos son un subconjunto “optimizado”



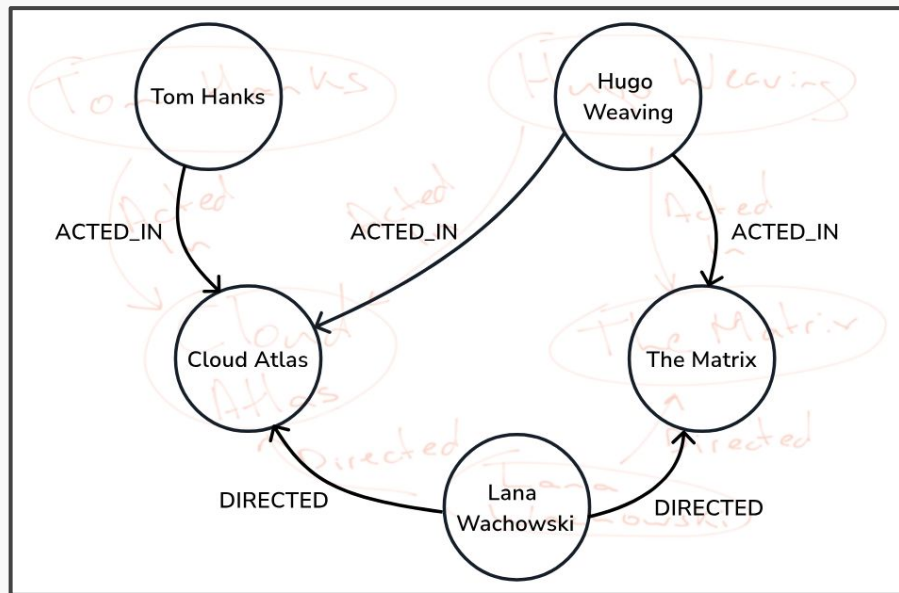
Bases de datos de grafos



Amazon Neptune

Pre: Modelo de una base de datos de grafos

- Conjunto de nodos V
- Conjunto de aristas $E \subseteq V \times V$
- Nodos y aristas pueden tener atributos
- Consultas en base a patrones



Neo4j



- Model Property Graph
- Cypher

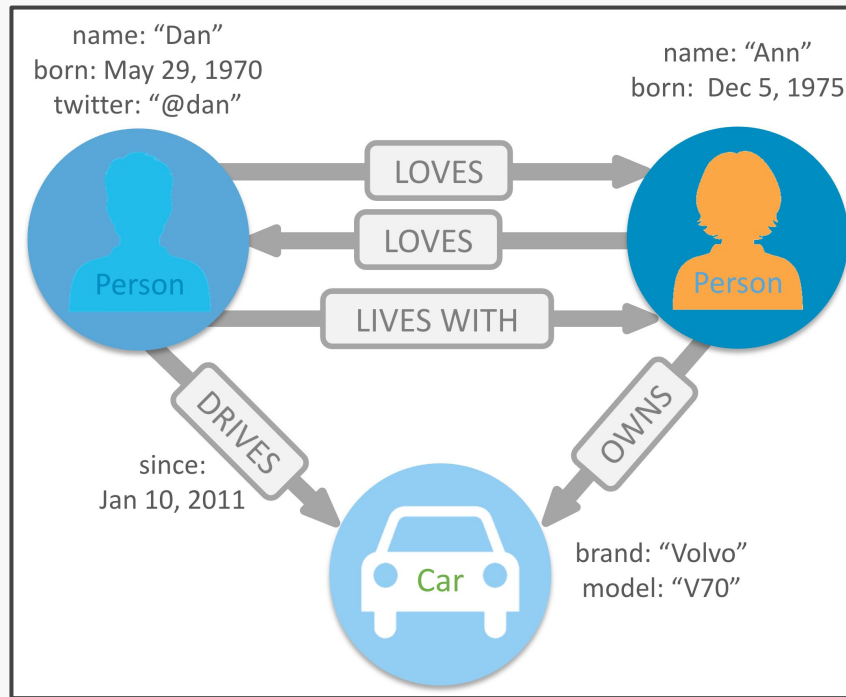
Agenda

✨ **Neo4j: Property Graph Model**

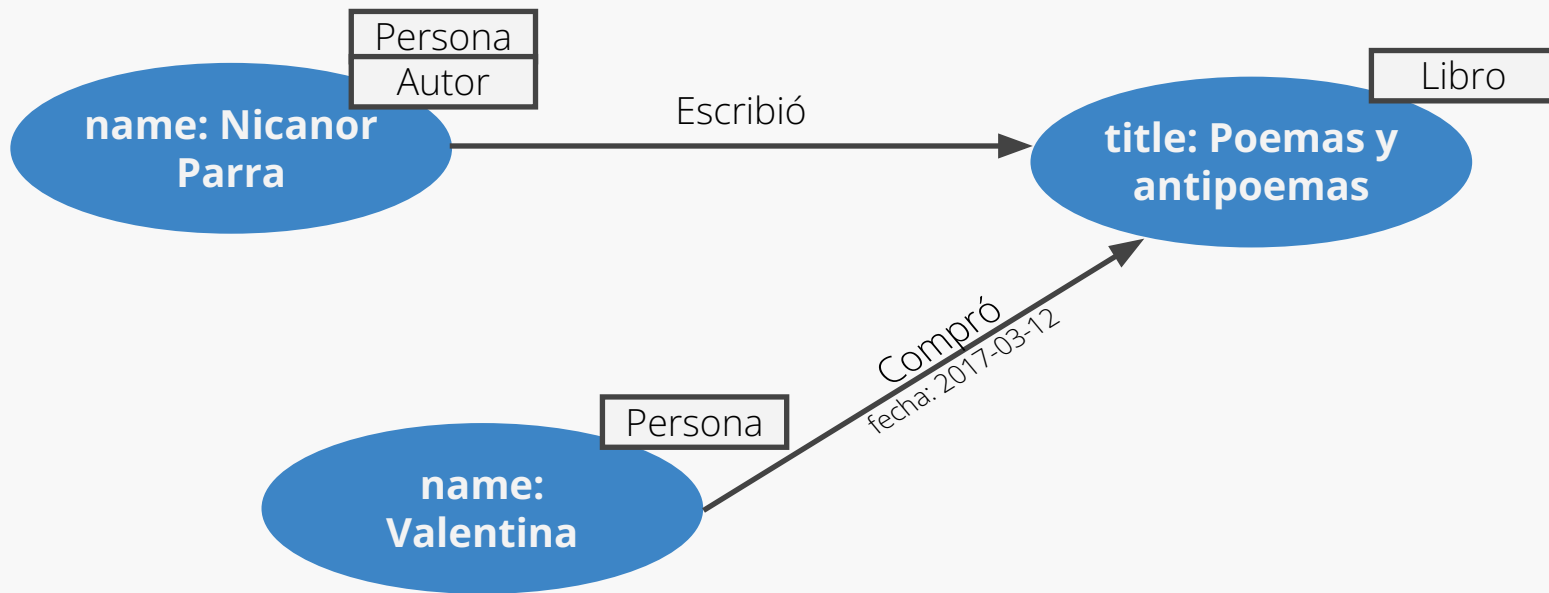
Neo4j: Cypher Query Language

Modelo Property Graph

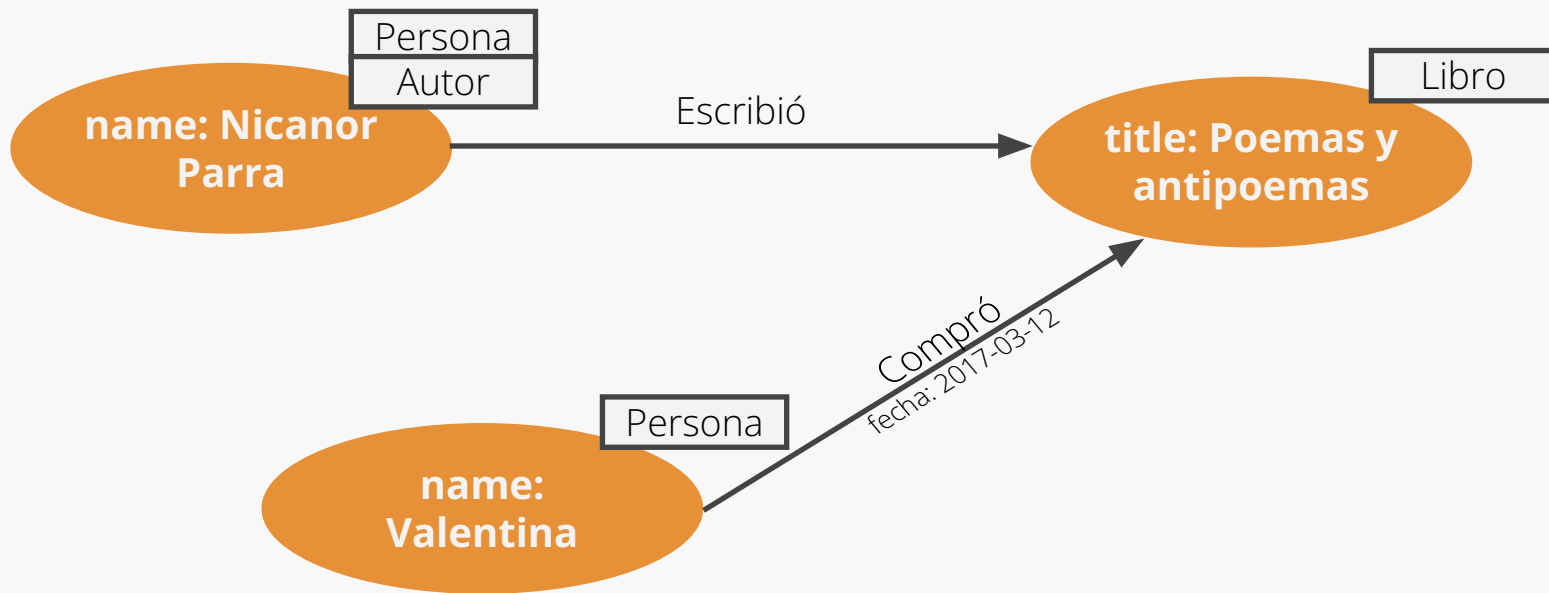
- Grafo
- Nodos (entidades)
- Arcos (relaciones)
- Nodos y arcos tienen etiqueta
- Nodos y arcos tienen atributos
- ¡No pueden haber links rotos!



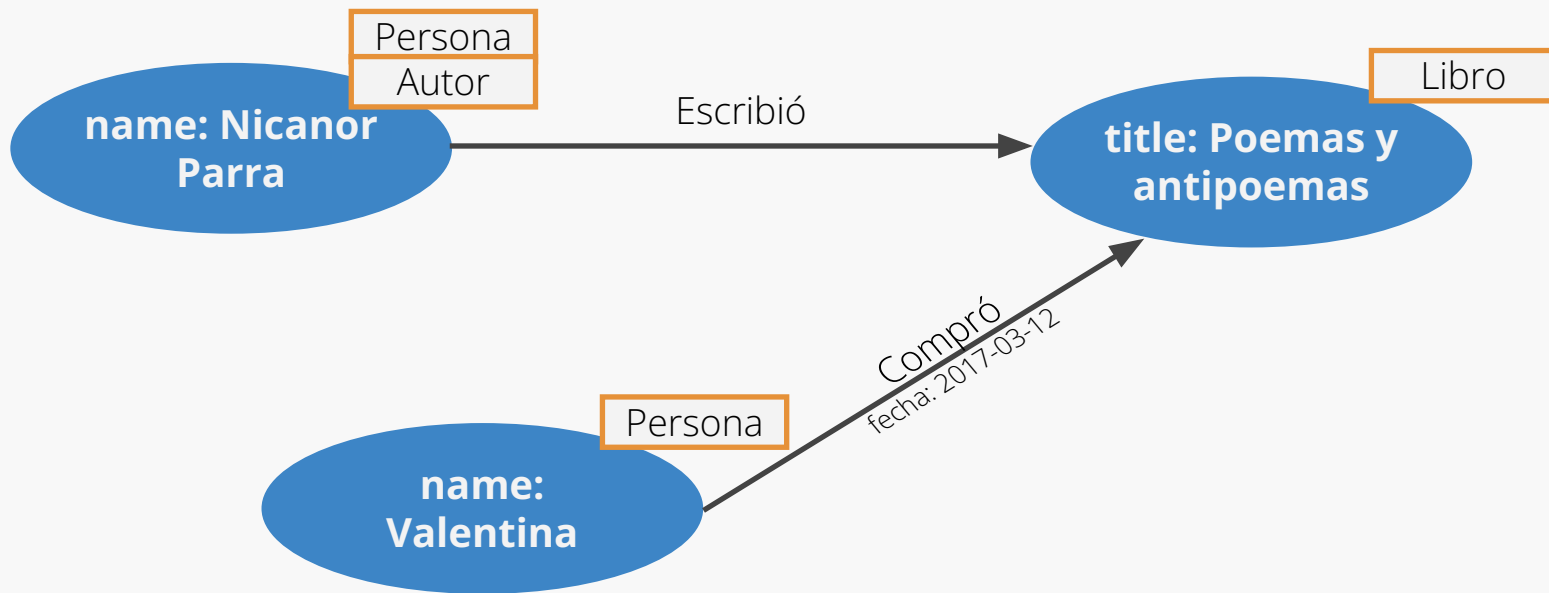
Modelo Property Graph



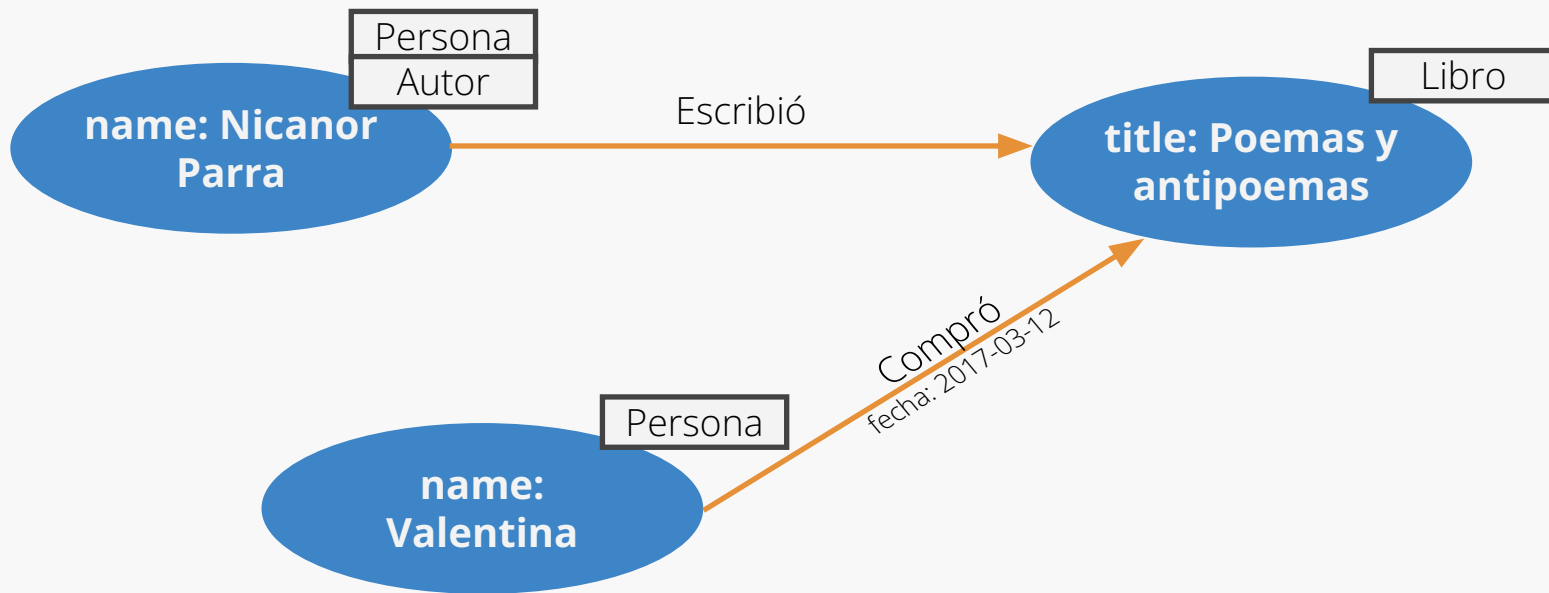
Modelo Property Graph: nodos



Modelo Property Graph: tipos de nodos



Modelo Property Graph: relaciones



Modelo Property Graph: atributos

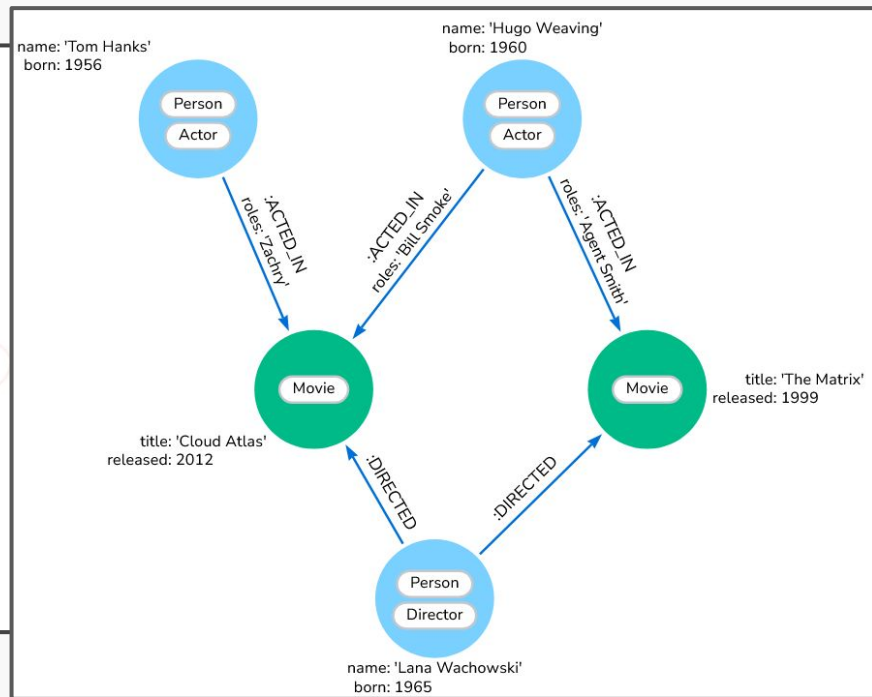
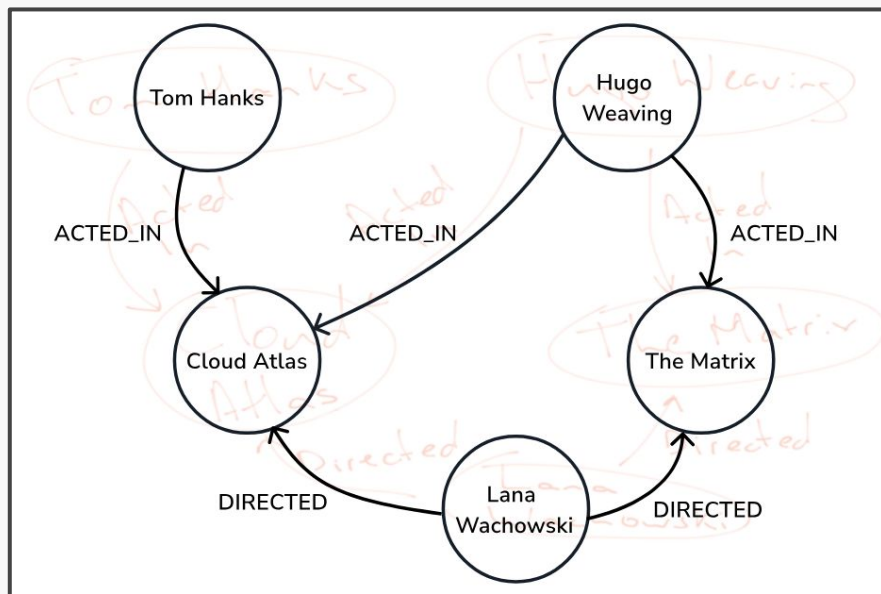


Figura: [Graph modeling guidelines - Getting Started](#)

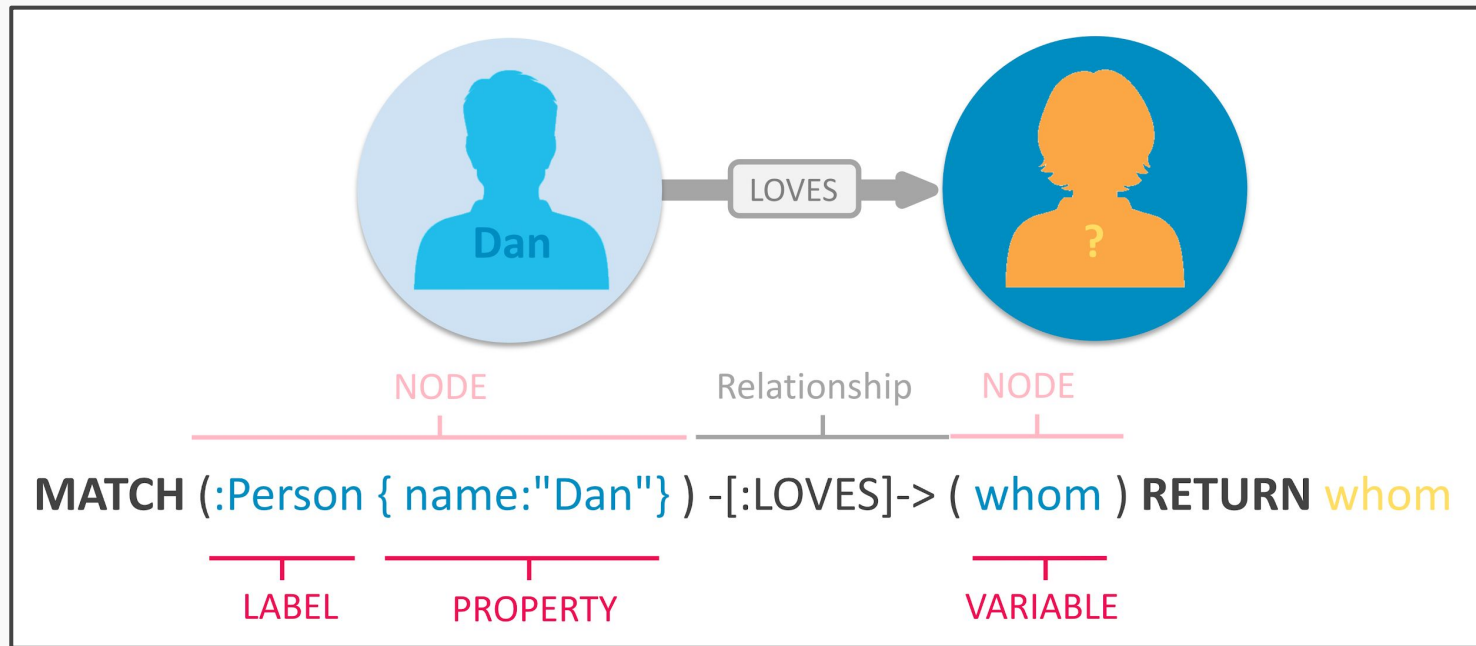
Agenda

Neo4j: Property Graph Model

✨ **Neo4j: Cypher Query Language**

Cypher

Lenguaje de consulta de Neo4j, basado en patrones



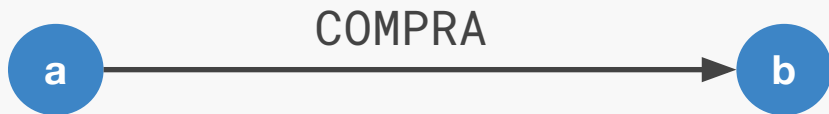
Cypher: maneras de especificar nodos

- `()` es cualquier nodo
- `(:Persona)` es cualquier nodo con el *label* (etiqueta) `Persona`
- `(a)` nombra a un nodo usando la variable `a`
- `(a:Persona)` nombra a un nodo con el *label* `Persona` usando la variable `a`

MATCH `(:Persona) --> (p:Persona)`

Cypher: maneras de especificar nodos

En este ejemplo construimos un patrón que describe la relación entre dos nodos cualquiera (que pondremos en las variables a y b, respectivamente) pero que debe ser el tipo COMPRA. Notar que la dirección de la relación importa (por eso usamos una flecha)



`(a) -[:COMPRA]-> (b)`

Cypher: maneras de especificar relaciones

En Cypher, las relaciones las representamos por medio de una flecha:

- `-->` indica que existe una relación de un nodo a otro
- `-[:ETIQUETA]->` indica que existe una relación con una etiqueta
- `-[:ETIQUETA1 | :ETIQUETA2]->` existe una relación o la otra
- `-[{since:2010}]->` una relación con el valor 2010 en la propiedad since
- `-[:KNOWS*..4]->` un camino de largo 4 o menos por relaciones KNOWS

Cypher: relaciones en variables

Si queremos acceder a información sobre una relación, al igual que con los nodos, podemos guardar la relación en una variable:

```
-[rel:ETIQUETA]->
```

```
-[rel]->
```

Cypher: escribiendo un patrón paso a paso

1. **() --> ()**
cualquier relación
2. **(a) --> (b)**
lo mismo, pero los nodos quedan en variables a y b
3. **(a) -[:COMPRA]-> (b)**
lo mismo, pero solo relaciones COMPRA
4. **(a:PERSONA) -[:COMPRA]-> (b:LIBRO)**
lo mismo pero indicamos el tipo de nodos
5. **(a:PERSONA) -[r:COMPRA]-> (b:LIBRO)**
es lo mismo pero la relación queda en la variable r

Cypher: ejecutando una consulta

Lo mínimo en una consulta es **MATCH** y **RETURN**

```
MATCH (node:Label1) RETURN node.property
```

Podemos usar las variables en **WHERE** para condiciones adicionales

```
MATCH (node1:Label1)--> (node2:Label2)  
WHERE node1.propertyA = {value}  
RETURN node2.propertyA, node2.propertyB
```

Cypher: repaso

Este es un ejemplo en Cypher de una consulta que nos permite filtrar a nivel de nodo:

1. El patrón (**MATCH**) indica una relación de un nodo de tipo Label1 a otro de tipo Label2
2. El filtro (**WHERE**) agrega una condición sobre el valor de propertyA del nodo1
3. Sobre los valores obtenidos, queremos retornar (**RETURN**) dos propiedades del nodo2:

```
MATCH (node1:Label1) --> (node2:Label2)
WHERE node1.propertyA = {value}
RETURN node2.propertyA, node2.propertyB
```


Cypher

¡Vamos a practicar!

Agenda

Neo4j: Property Graph Model

Neo4j: Cypher Query Language

✨ **Más consultas**

Cypher: algunos patrones interesantes

- Friend-of-a-friend (amigo de un amigo):

```
(user)-[:KNOWS]-(friend)-[:KNOWS]-(foaf)
```

- Camino más corto entre dos nodos:

```
path = shortestPath( (user)-[:KNOWS*..5]-(other) )
```

- Filtrado colaborativo (recomendación básica):

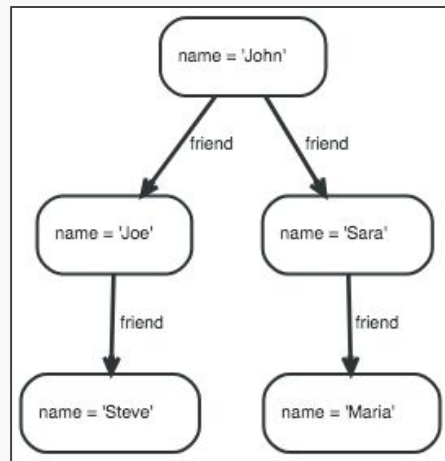
```
(user)-[:PURCHASED]->(product)<-[:PURCHASED]-( )-[:PURCHASED]->(otherProduct)
```

Cypher: evaluando consultas

En la siguiente consulta, queremos todos los amigos de un amigo de John:

```
MATCH (john {name: 'John'})-[:friend]->()-[:friend]->(fof)  
RETURN john.name, fof.name
```

john.name	fof.name
"John"	"Maria"
"John"	"Steve"

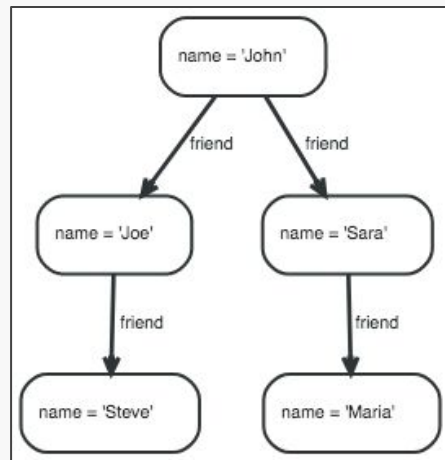


Cypher: evaluando consultas

En la siguiente consulta, queremos todas las personas que tienen a un amigo cuyo nombre empieza con la letra "S":

```
MATCH (user)-[:friend]->(follower)
WHERE user.name IN ['Joe', 'John', 'Sara', 'Maria', 'Steve'] AND follower.name =~ 'S.*'
RETURN user.name, follower.name
```

user.name	follower.name
"Joe"	"Steve"
"John"	"Sarah"



Cypher: múltiples patrones

Podemos utilizar múltiples **MATCH** en la misma consulta para calcular el producto cartesiano entre los resultados. En este caso, todos los resultados del primer patrón se “cruzarán” con los del segundo, formando todos los pares posibles, y esas variables se usarán para formar el valor de retorno

```
MATCH (user:Person) WITH user LIMIT 20
MATCH (user)-[:friend]->(follower)
WHERE follower.name IN ['Joe', 'John', 'Sara', 'Maria', 'Steve']
RETURN user.name, follower.name
```

En el ejemplo tomamos 20 usuarios y los relacionamos con aquellos amigos cuyo nombre esté en la lista entregada. Posteriormente, se mostrará una tabla con todos los pares usuario-amigo resultantes

Cypher: patrones opcionales

Al igual que **MATCH**, **OPTIONAL MATCH** busca patrones en un grafo. La diferencia está en que si **OPTIONAL MATCH** no encuentra coincidencias para su patrón, las partes (variables) de ese patrón usarán el valor **null**

```
MATCH (user:Person)
OPTIONAL MATCH (user)-[:friend]->(follower)
RETURN user.name, follower.name
```

Este ejemplo retornará una tabla con todos los pares usuario-amigo para todas las personas. Si una persona no tiene amigos, aparecerá en la tabla, pero con un valor null en la segunda columna

Cypher: creando nodos

Para crear un nodo o relación usamos **CREATE**. Recordar que en Neo4j no pueden existir relaciones “sueltas” (toda relación ocurre entre dos nodos), por lo que primero agregamos un nodo y luego agregamos sus relaciones con otros:

- Ejemplo de creación de un nodo persona

```
CREATE (nodo:Person {name:"Antonio Ossa"})  
RETURN nodo
```

- Ejemplo de creación de un arco indicando dónde trabaja la persona

```
MATCH (nodo:Person {name:"Antonio Ossa"})  
CREATE (nodo)-[trabajo:TRABAJA_EN]->(neo:Database {name:"PUC" })  
RETURN nodo
```


Cypher: creando muchos nodos

En este ejemplo, ocupamos **FOREACH** para iterar sobre un conjunto de valores y realizar una operación sobre cada uno de ellos.

```
MATCH (nodo:Person {name:"Antonio Ossa"})
FOREACH (nombre in ["Francisca","Dante"] |
  CREATE (nodo)-[:FRIEND]->(:Person {name:nombre}))
```

En particular, la consulta dice “toma los nodos de tipo persona que tienen el valor Antonio Ossa en la property nombre, recorre cada nombre en el arreglo para crear una relación de amistad (FRIEND) entre el nodo y una persona con el nombre entregado”

Cypher: eliminando nodos

Por último, Cypher nos permite eliminar nodos utilizando **DELETE**, donde también podemos ocupar **MATCH** para definir un patrón y luego indicar la operación a realizar sobre la variable utilizada:

```
MATCH (nodo:Person {name:"Antonio Ossa"})  
DELETE nodo
```

En Neo4j no se pueden borrar nodos que tengan relaciones conectadas a ellos, por lo que primero corresponde eliminar las relaciones y luego se puede eliminar el nodo. Podemos hacer todo esto usando **DETACH DELETE** en lugar de solo **DELETE**:

```
MATCH (nodo:Person {name:"Antonio Ossa"})  
DETACH DELETE nodo
```

Agenda

Neo4j: Property Graph Model

Neo4j: Cypher Query Language

Más consultas



Extra: cargar CSVs

Cómo cargar un archivo CSV

En múltiples ocasiones, nos encontraremos con que los datos que queremos utilizar para construir nuestro grafo en Neo4j vienen en formato CSV. Supongamos que tenemos un archivo llamado `users.csv` con los nodos que queremos cargar.

Para importar el archivo debemos utilizar el comando [LOAD CSV](#), con el que podremos indicar que el archivo tiene una fila con los nombres de las columnas (WITH HEADERS) y, luego, cómo usar la información de cada fila para crear los nodos

```
user_id,display_name
604,schernichkin
605,Damian Grzanka
606,sandrooco
607,Bobis
608,wanderingcatto
609,bguegan
```

Archivo `users.csv`

```
LOAD CSV WITH HEADERS
FROM 'file:///users.csv' AS row
MERGE (u:User {userId: row.user_id, name: row.display_name})
RETURN row
```

Consulta para cargar cada fila como un nodo

Cómo cargar un archivo CSV

En cambio, si tuviéramos un archivo llamado `asked.csv` con los arcos que relacionan dos nodos en nuestro grafo, la consulta para cargar los arcos cambia levemente

Al igual que en el caso anterior, utilizamos el comando [LOAD CSV](#), pero ahora tendremos que usar `MATCH` antes de hacer `MERGE` para seleccionar los nodos que queremos unir por medio de un arco (de acuerdo a la información en cada fila del archivo CSV)

```
start_id,end_id
604,4622
605,4623
606,4624
607,4625
608,4626
609,4627
```

Archivo `asked.csv`

```
LOAD CSV WITH HEADERS
FROM 'file:///asked.csv' AS row
MATCH (u:User {userId: row.start_id})
MATCH (q:Question {questionId: row.end_id})
MERGE (u) -[:ASKED]-> (q);
RETURN *
```

Consulta para cargar cada fila como un arco

Clase 3.1

Neo4j

Teoría de Grafos para Machine Learning

Antonio Ossa Guerra
aaossa@ing.puc.cl